Spring MVC Security

In this lesson, we're going to explore the critical subject of Spring Security. This section aims to give you an all-encompassing overview and practical understanding of how to secure a Spring MVC web application.

Overview of Topics:

- Development of login pages
- Usage of default and custom pages
- Definition of users and roles with simple authentication
- Protection of URLs based on specific roles
- Hiding and showing content based on roles
- Storing user passwords and roles in databases, in both plain text and encrypted forms

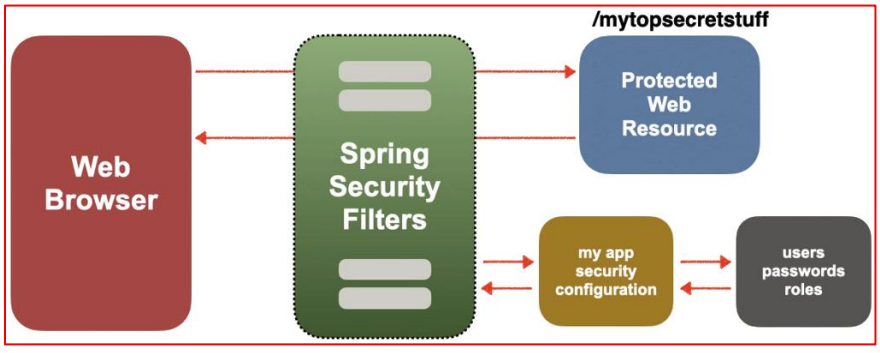
Introduction to Spring Security: This lesson about Spring Security is not an A to Z reference but a practical approach that covers the most common daily tasks. For a more exhaustive exploration, please consult the Spring Security Reference Manual. You can access the manual by visiting the official Spring Reference Manual (`docs.spring.io/spring-security/reference`).

Spring Security Model:

- **Definition**: Spring Security defines a framework for security implemented using Servlet filters.
- **Servlet Filters**: These filters pre-process and post-process web requests, routing them based on security logic. Spring provides the bulk of the security functionality with Servlet filters.

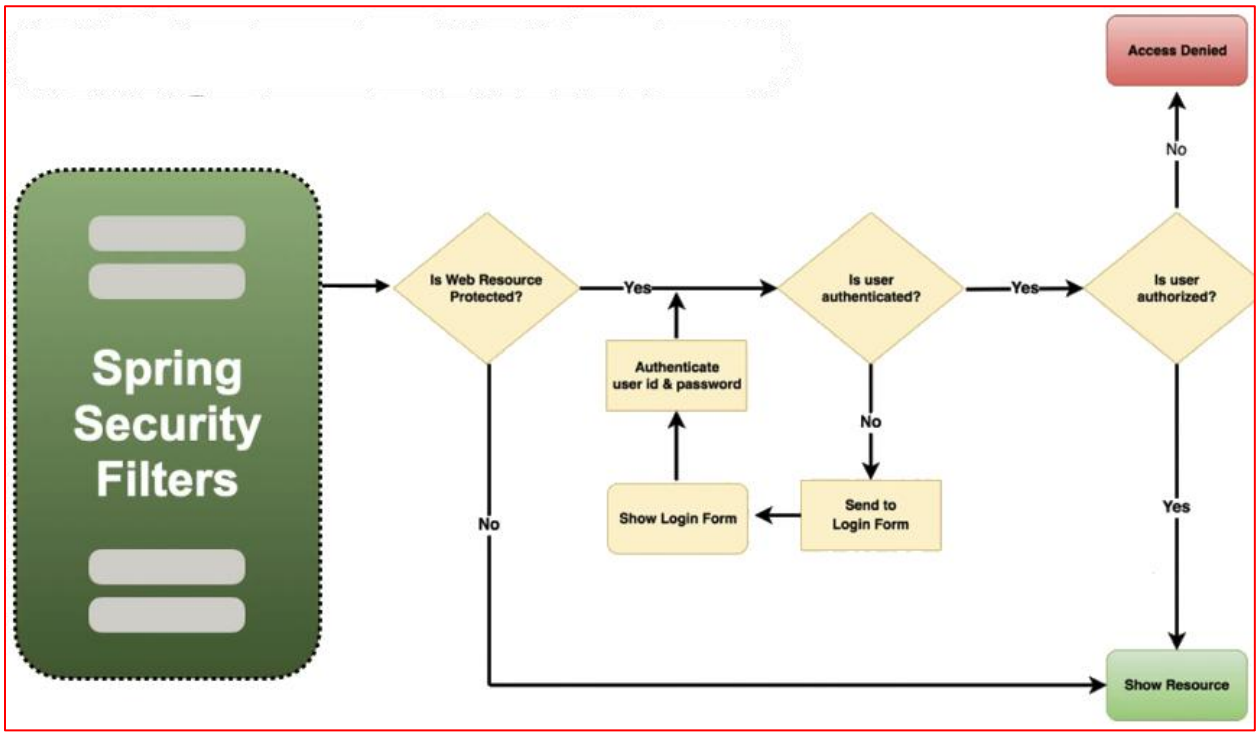
How Spring Security Works:

1. **Web Browser and Protected Web Resource**: When the browser attempts to access protected information, Spring Security intercepts the request.
2. **Authentication and Authorization**: Spring Security examines the application's security configuration and checks the user's passwords and roles in the database to authenticate and authorize access.



Spring Security in Action – Flow Chart:

1. **Authentication**: Is this web resource protected? Is the user authenticated? If not, redirect to the login form and validate credentials.
2. **Authorization**: Check if the authenticated user is authorized to access the resource. Access depends on the security badge, similar to access levels in a building at a job or university.
3. **Access Granted or Denied**: If the user has the given role, access is granted; otherwise, it's denied.



Security Concepts:

- **Authentication**: Checking the user ID and password against stored credentials.
- **Authorization**: Checking if the user has an authorized role.
- **Declarative Security**: Defining application security constraints in a configuration using an all-Java config.
- **Programmatic Security**: Spring provides an API for custom application coding for greater customization.

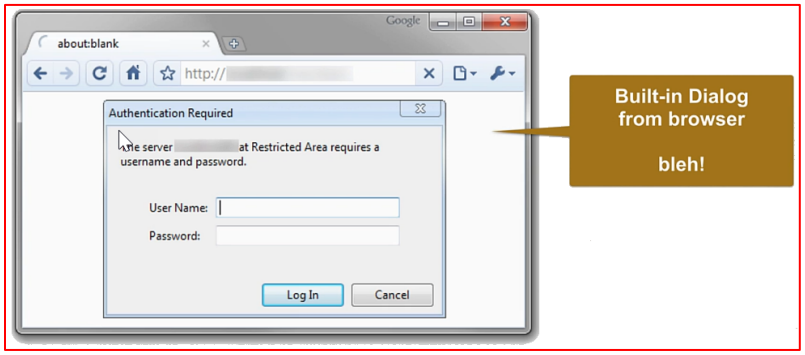
Real-World Application: Spring Security can be easily extended to suit additional business rules or specific logic in your company.

Enabling Spring Security:

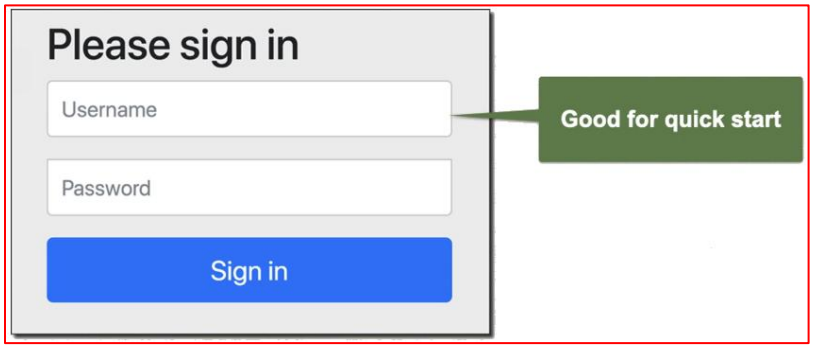
1. **Edit the pom.xml File**: Add the dependency for spring-boot-starter-security.
2. **Automatic Security**: By adding this dependency, all endpoints for the application are secured, with customization options available later.

Login Options:

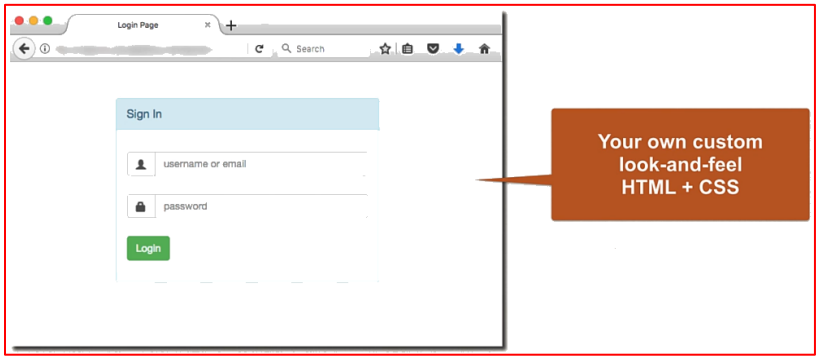
- **HTTP Basic Authentication**: A standard browser pop-up for credentials (though considered "blah" or ugly).



- **Default Login Form**: Spring's built-in login form.



- **Custom Login Form**: Create your own form with HTML, CSS, and JavaScript.



Final Project Demonstration: For our project, we will build a custom login page with access based on different roles such as employee, manager, and admin. Special pages will be available only to specific roles.

